Pizza Sales Analysis Using Python

Tools: Python, pandas, Matplotlib, Seaborn
Focus: Order-level sales analysis and product insights

Business Problem

The goal of this project is to analyze pizza sales data to understand:

- Customer ordering behavior
- Product performance
- Time-based sales trends

All metrics are calculated at the **order level** to avoid common aggregation errors.

Data Understanding

Each row in the dataset represents a **pizza line item**, not a complete order. As multiple pizzas can appear within the same order, aggregation by `Order_id` is required to avoid incorrect averages and inflated metrics.

Feature Engineering

Additional columns were created to support time-based analysis and proper sorting:

- `order_hour` for hourly ordering patterns
- `day_number` to enforce correct weekday order
- `month_number` to prevent alphabetical month sorting errors

In [9]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(
    style="whitegrid",
    palette="Set2",
    font="sans-serif",
    font_scale=1.15
)

plt.rcParams.update({
    "figure.figsize": (10, 6),
    "axes.titleweight": "bold",
```

```
    "axes.titlesize": 15,
    "axes.labelsize": 12
})
```

In [10]:
```
df = pd.read_excel(
    "C:/Users/IsaacOgbaji/Downloads/pizza_sales.xlsx"
)

df.head()
```

Out[10]:

	Pizza_id	Order_id	Pizza_name_id	Quantity	Order_date	Order_time	Unit_price	Total_p
0	1	1	hawaiian_m	1	2015-01-01	11:38:36	13.25	1
1	2	2	classic_dlx_m	1	2015-01-01	11:57:40	16.00	1
2	3	2	five_cheese_l	1	2015-01-01	11:57:40	18.50	1
3	4	2	ital_supr_l	1	2015-01-01	11:57:40	20.75	2
4	5	2	mexicana_m	1	2015-01-01	11:57:40	16.00	1

In [11]:
```
df["order_hour"] = df["Order_date"].dt.hour
df["day_number"] = df["Order_date"].dt.weekday + 1
df["month_number"] = df["Order_date"].dt.month
```

Key Metrics

The following key performance indicators (KPIs) were calculated using order-level aggregation:

- **Total Revenue**
- **Total Orders (Distinct Order IDs)**
- **Average Order Value (AOV)**
- **Average Quantity per Order**

All metrics are validated using totals and distinct counts.

In [ ]:
```
total_revenue = df["Total_price"].sum()

total_orders = df["Order_id"].nunique()

avg_order_value = total_revenue / total_orders

order_summary = (
    df.groupby("Order_id", as_index=False)
      .agg(total_quantity=("Quantity", "sum"))
)

avg_quantity_per_order = order_summary["total_quantity"].mean()

total_revenue, total_orders, avg_order_value, avg_quantity_per_order
```

In [6]:
```
import pandas as pd
import numpy as np
```

In [2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

In [70]:
```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(
    style="whitegrid",
    palette="Set2",
    font_scale=1.1
)

plt.rcParams["figure.figsize"] = (10, 6)
plt.rcParams["axes.titleweight"] = "bold"
plt.rcParams["axes.titlesize"] = 14
plt.rcParams["axes.labelsize"] = 12
```

In [10]:
```
df.shape
df.info()
df.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 48620 entries, 0 to 48619
Data columns (total 12 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Pizza_id          48620 non-null  int64
 1   Order_id          48620 non-null  int64
 2   Pizza_name_id     48620 non-null  object
 3   Quantity          48620 non-null  int64
 4   Order_date        48620 non-null  datetime64[ns]
 5   Order_time        48620 non-null  object
 6   Unit_price        48620 non-null  float64
 7   Total_price       48620 non-null  float64
 8   Pizza_size        48620 non-null  object
 9   Pizza_category    48620 non-null  object
 10  Pizza_ingredients 48620 non-null  object
 11  Pizza_name        48620 non-null  object
dtypes: datetime64[ns](1), float64(2), int64(3), object(6)
memory usage: 4.5+ MB
```

Out[10]:

	Pizza_id	Order_id	Pizza_name_id	Quantity	Order_date	Order_time	Unit_price	Total_p
0	1	1	hawaiian_m	1	2015-01-01	11:38:36	13.25	1
1	2	2	classic_dlx_m	1	2015-01-01	11:57:40	16.00	1
2	3	2	five_cheese_l	1	2015-01-01	11:57:40	18.50	1
3	4	2	ital_supr_l	1	2015-01-01	11:57:40	20.75	2
4	5	2	mexicana_m	1	2015-01-01	11:57:40	16.00	1

In [12]:

```
df.head(10)
```

Out[12]:

	Pizza_id	Order_id	Pizza_name_id	Quantity	Order_date	Order_time	Unit_price	Total_p
0	1	1	hawaiian_m	1	2015-01-01	11:38:36	13.25	1
1	2	2	classic_dlx_m	1	2015-01-01	11:57:40	16.00	1
2	3	2	five_cheese_l	1	2015-01-01	11:57:40	18.50	1
3	4	2	ital_supr_l	1	2015-01-01	11:57:40	20.75	2
4	5	2	mexicana_m	1	2015-01-01	11:57:40	16.00	1
5	6	2	thai_ckn_l	1	2015-01-01	11:57:40	20.75	2
6	7	3	ital_supr_m	1	2015-01-01	12:12:28	16.50	1
7	8	3	prsc_argla_l	1	2015-01-01	12:12:28	20.75	2
8	9	4	ital_supr_m	1	2015-01-01	12:16:31	16.50	1
9	10	5	ital_supr_m	1	2015-01-01	12:21:30	16.50	1

In [13]:
```
df.shape
```

Out[13]: (48620, 12)

In [2]:
```
import pandas as pd
```

In [4]:
```
import pandas as pd
import numpy as np
```

In [5]:
```
df = pd.read_excel("C:/Users/IsaacOgbaji/Downloads/pizza_sales.xlsx")
```

In [6]:
```
type(df)
```

Out[6]:
```
pandas.core.frame.DataFrame
```

In [7]:
```
df.head()
```

Out[7]:

	Pizza_id	Order_id	Pizza_name_id	Quantity	Order_date	Order_time	Unit_price	Total_p
0	1	1	hawaiian_m	1	2015-01-01	11:38:36	13.25	1
1	2	2	classic_dlx_m	1	2015-01-01	11:57:40	16.00	1
2	3	2	five_cheese_l	1	2015-01-01	11:57:40	18.50	1
3	4	2	ital_supr_l	1	2015-01-01	11:57:40	20.75	2
4	5	2	mexicana_m	1	2015-01-01	11:57:40	16.00	1

In [8]:
```
df.shape
```

Out[8]:
```
(48620, 12)
```

In [10]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 48620 entries, 0 to 48619
Data columns (total 12 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Pizza_id          48620 non-null  int64
 1   Order_id          48620 non-null  int64
 2   Pizza_name_id     48620 non-null  object
 3   Quantity          48620 non-null  int64
 4   Order_date        48620 non-null  datetime64[ns]
 5   Order_time        48620 non-null  object
 6   Unit_price        48620 non-null  float64
 7   Total_price       48620 non-null  float64
 8   Pizza_size        48620 non-null  object
 9   Pizza_category    48620 non-null  object
 10  Pizza_ingredients 48620 non-null  object
 11  Pizza_name        48620 non-null  object
dtypes: datetime64[ns](1), float64(2), int64(3), object(6)
memory usage: 4.5+ MB
```

In [13]:
```
df["Order_date"] = pd.to_datetime(df["Order_date"])
```

In [14]:
```
df["Order_time"] = pd.to_datetime(df["Order_time"], format="%H:%M:%S").dt.time
```

In [16]:
```
total_rows = len(df)
unique_orders = df["Order_id"].nunique()

total_rows, unique_orders
```

Out[16]:
```
(48620, 21350)
```

Total Revenue

In [18]:
```
total_revenue = df["Total_price"].sum()
total_revenue
```

Out[18]:
```
np.float64(817860.0499999999)
```

Total Orders

In [19]:
```
total_orders = df["Order_id"].nunique()
total_orders
```

Out[19]:
```
21350
```

using this method wont be correct

In [21]:
```
df["Quantity"].mean()
```

Out[21]:
```
np.float64(1.0196215549156726)
```

best method to arrive at the avearge is to create Order Level table

In [22]:
```
order_summary = (
    df
    .groupby("Order_id", as_index=False)
    .agg(
        total_quantity=("Quantity", "sum"),
        total_revenue=("Total_price", "sum")
    )
)
```

AVERAGE QUANTITY PER ORDER

In [24]:
```
avg_quantity_per_order = order_summary["total_quantity"].mean()
avg_quantity_per_order
```

Out[24]:
```
np.float64(2.321967213114754)
```

Let's verify with totals

In [26]:
```
df["Quantity"].sum() / df["Order_id"].nunique()
```

Out[26]:
```
np.float64(2.321967213114754)
```

AVERAGE ORDER VALUE (AOV)

In [27]:
```
avg_order_value = order_summary["total_revenue"].mean()
avg_order_value
```

Out[27]:
```
np.float64(38.30726229508197)
```

OR

In [28]:
```
total_revenue / total_orders
```

Out[28]:
```
np.float64(38.30726229508196)
```

TIME-BASED ANALYSIS

In [ ]:
```
Order Hour
```

In [31]:
```
df["order_hour"] = pd.to_datetime(df["Order_time"], format="%H:%M:%S").dt.hour
```

Day of Week (Name)

In [32]:
```
df["day_of_week"] = df["Order_date"].dt.day_name()
```

Day Number (FOR SORTING)

In [33]:
```
df["day_number"] = df["Order_date"].dt.weekday + 1
```

Month Name

In [34]:
```
df["month"] = df["Order_date"].dt.month_name()
```

Month Number (FOR SORTING)

In [35]:
```
df["month_number"] = df["Order_date"].dt.month
```

MONTHLY REVENUE TREND

In [37]:
```
df.columns
```

Out[37]:
```
Index(['Pizza_id', 'Order_id', 'Pizza_name_id', 'Quantity', 'Order_date',
       'Order_time', 'Unit_price', 'Total_price', 'Pizza_size',
       'Pizza_category', 'Pizza_ingredients', 'Pizza_name', 'order_date',
       'order_hour', 'day_of_week', 'day_number', 'month', 'month_number'],
      dtype='object')
```

In [38]:
```
day_revenue = (
    df.groupby(["day_number", "day_of_week"], as_index=False)
      .agg(total_revenue=("Total_price", "sum"))
      .sort_values("day_number")
)
```

In [39]:
```
monthly_revenue = (
    df.groupby(["month_number", "month"], as_index=False)
      .agg(total_revenue=("Total_price", "sum"))
      .sort_values("month_number")
)
```

ORDER VOLUME BY HOUR

Order count by hour

In [40]:
```
hourly_orders = (
    df.groupby("order_hour", as_index=False)
      .agg(total_orders=("Order_id", "nunique"))
)
```

In [41]:
```
hourly_orders.sort_values("total_orders", ascending=False).head()
```

Out[41]:

	order_hour	total_orders
3	12	2520
4	13	2455
9	18	2399
8	17	2336
10	19	2009

PRODUCT ANALYSIS

In [42]:
```
df.columns
```

Out[42]:
```
Index(['Pizza_id', 'Order_id', 'Pizza_name_id', 'Quantity', 'Order_date',
       'Order_time', 'Unit_price', 'Total_price', 'Pizza_size',
       'Pizza_category', 'Pizza_ingredients', 'Pizza_name', 'order_date',
       'order_hour', 'day_of_week', 'day_number', 'month', 'month_number'],
      dtype='object')
```

TOP-SELLING PIZZAS (BY QUANTITY)

Product Analysis

This section explores product performance across individual pizzas, categories, and sizes to understand customer preferences and revenue drivers.

In [43]:
```
top_pizzas = (
    df.groupby("Pizza_name", as_index=False)
      .agg(
          total_quantity=("Quantity", "sum"),
          total_revenue=("Total_price", "sum"),
          total_orders=("Order_id", "nunique")
      )
      .sort_values("total_quantity", ascending=False)
)
```

Top 10 Best-Selling Pizzas

In [44]:
```
top_10_pizzas = top_pizzas.head(10)
top_10_pizzas
```

Out[44]:

	Pizza_name	total_quantity	total_revenue	total_orders
7	The Classic Deluxe Pizza	2453	38180.50	2329
0	The Barbecue Chicken Pizza	2432	42768.00	2273
12	The Hawaiian Pizza	2422	32273.25	2280
20	The Pepperoni Pizza	2418	30161.75	2278
30	The Thai Chicken Pizza	2371	43434.25	2225
4	The California Chicken Pizza	2370	41409.50	2197
23	The Sicilian Pizza	1938	30940.50	1820
26	The Spicy Italian Pizza	1924	34831.25	1822
25	The Southwest Chicken Pizza	1917	34705.75	1825
1	The Big Meat Pizza	1914	22968.00	1811

These pizzas drive the highest sales volume and should be prioritized for menu positioning, inventory planning, and promotions.

PIZZA CATEGORY ANALYSIS

In [45]:
```
category_analysis = (
    df.groupby("Pizza_category", as_index=False)
      .agg(
          total_revenue=("Total_price", "sum"),
          total_quantity=("Quantity", "sum"),
          total_orders=("Order_id", "nunique")
      )
      .sort_values("total_revenue", ascending=False)
)
```

In [46]:
```
category_analysis
```

Out[46]:

	Pizza_category	total_revenue	total_quantity	total_orders
1	Classic	220053.10	14888	10859
2	Supreme	208197.00	11987	9085
0	Chicken	195919.50	11050	8536
3	Veggie	193690.45	11649	8941

PIZZA SIZE ANALYSIS

What pizza sizes do customers prefer?

In [48]:
```
size_analysis = (
    df.groupby("Pizza_size", as_index=False)
      .agg(
          total_orders=("Order_id", "nunique"),
          total_quantity=("Quantity", "sum"),
          total_revenue=("Total_price", "sum")
      )
      .sort_values("total_orders", ascending=False)
)
```

In [49]:
```
size_analysis
```

Out[49]:

	Pizza_size	total_orders	total_quantity	total_revenue
0	L	12736	18956	375318.70
1	M	11159	15635	249382.25
2	S	10490	14403	178076.50
3	XL	544	552	14076.00
4	XXL	28	28	1006.60

TOP PIZZAS BY MONTH

Aggregate by Month + Pizza

In [51]:
```
pizza_monthly = (
    df.groupby(
        ["month_number", "month", "Pizza_name"],
        as_index=False
    )
    .agg(
        total_quantity=("Quantity", "sum"),
        total_revenue=("Total_price", "sum")
    )
    .sort_values(["month_number", "total_quantity"], ascending=[True, False])
)
```

Top 5 Pizzas per Month

In [52]:
```
top_pizzas_by_month = (
    pizza_monthly
    .groupby("month")
    .head(5)
)
```

PIZZA CATEGORY PERFORMANCE BY MONTH

In [53]:
```
category_monthly = (
    df.groupby(
        ["month_number", "month", "Pizza_category"],
        as_index=False
    )
    .agg(
        total_revenue=("Total_price", "sum"),
        total_quantity=("Quantity", "sum"),
        total_orders=("Order_id", "nunique")
    )
    .sort_values("month_number")
)
```

In [54]:
```
category_monthly
```

Out[54]:

	month_number	month	Pizza_category	total_revenue	total_quantity	total_orders
0	1	January	Chicken	16188.75	913	714
1	1	January	Classic	18619.40	1257	920
2	1	January	Supreme	17929.75	1044	779
3	1	January	Veggie	17055.40	1018	774
4	2	February	Chicken	15468.25	875	674
5	2	February	Classic	17336.10	1178	871
6	2	February	Supreme	16718.85	964	720
7	2	February	Veggie	15636.40	944	723
8	3	March	Chicken	17625.50	994	764
9	3	March	Classic	18116.60	1236	917
10	3	March	Supreme	17319.65	991	767
11	3	March	Veggie	17335.35	1040	795
12	4	April	Chicken	16473.00	924	715
13	4	April	Classic	18816.80	1253	923
14	4	April	Supreme	17643.55	1013	777
15	4	April	Veggie	15803.45	961	739
16	5	May	Chicken	16752.25	939	735
17	5	May	Classic	19423.10	1324	939
18	5	May	Supreme	18315.65	1045	787
19	5	May	Veggie	16911.75	1020	806
20	6	June	Chicken	16086.50	910	706
21	6	June	Classic	18035.90	1199	865
22	6	June	Supreme	18097.25	1040	784
23	6	June	Veggie	16010.55	958	732
24	7	July	Chicken	17002.25	963	757
25	7	July	Classic	19653.00	1331	962
26	7	July	Supreme	18216.85	1041	815
27	7	July	Veggie	17685.80	1057	804
28	8	August	Chicken	16352.50	934	728
29	8	August	Classic	18821.90	1283	936

	month_number	month	Pizza_category	total_revenue	total_quantity	total_orders
30	8	August	Supreme	17093.45	991	755
31	8	August	Veggie	16010.40	960	765
32	9	September	Chicken	16039.00	900	707
33	9	September	Classic	17832.85	1202	867
34	9	September	Supreme	15295.60	877	671
35	9	September	Veggie	15012.60	911	689
36	10	October	Chicken	14792.00	832	639
37	10	October	Classic	17409.15	1181	865
38	10	October	Supreme	17275.10	998	746
39	10	October	Veggie	14551.35	872	655
40	11	November	Chicken	17399.75	981	738
41	11	November	Classic	18585.60	1262	948
42	11	November	Supreme	18202.80	1050	784
43	11	November	Veggie	16207.20	973	747
44	12	December	Chicken	15739.75	885	659
45	12	December	Classic	17402.70	1182	846
46	12	December	Supreme	16088.50	933	700
47	12	December	Veggie	15470.20	935	712

PIZZA SIZE TRENDS OVER TIME

In [55]:
```
size_monthly = (
    df.groupby(
        ["month_number", "month", "Pizza_size"],
        as_index=False
    )
    .agg(
        total_orders=("Order_id", "nunique"),
        total_quantity=("Quantity", "sum"),
        total_revenue=("Total_price", "sum")
    )
    .sort_values("month_number")
)
```

In [56]:
```
size_monthly
```

Out[56]:

	month_number	month	Pizza_size	total_orders	total_quantity	total_revenue
0	1	January	L	1098	1640	32399.40
1	1	January	M	938	1311	20943.50
2	1	January	S	898	1229	15103.50
3	1	January	XL	50	50	1275.00
4	1	January	XXL	2	2	71.90
5	2	February	L	1020	1521	30077.90
6	2	February	M	874	1210	19263.00
7	2	February	S	868	1189	14741.85
8	2	February	XL	38	38	969.00
9	2	February	XXL	3	3	107.85
10	3	March	L	1096	1651	32752.35
11	3	March	M	938	1313	20952.50
12	3	March	S	924	1253	15538.90
13	3	March	XL	41	41	1045.50
14	3	March	XXL	3	3	107.85
15	4	April	L	1032	1541	30622.70
16	4	April	M	992	1390	22176.75
17	4	April	S	843	1151	14136.05
18	4	April	XL	63	65	1657.50
19	4	April	XXL	4	4	143.80
20	5	May	L	1122	1668	32970.50
21	5	May	M	984	1338	21302.50
22	5	May	S	925	1265	15644.90
23	5	May	XL	54	54	1377.00
24	5	May	XXL	3	3	107.85
25	6	June	L	1097	1590	31493.80
26	6	June	M	941	1335	21367.00
27	6	June	S	816	1131	14048.00
28	6	June	XL	47	49	1249.50
29	6	June	XXL	2	2	71.90

	month_number	month	Pizza_size	total_orders	total_quantity	total_revenue
30	7	July	L	1164	1697	33583.05
31	7	July	M	1000	1396	22224.50
32	7	July	S	922	1249	15475.35
33	7	July	XL	50	50	1275.00
34	8	August	L	1053	1527	30267.90
35	8	August	M	951	1331	21175.50
36	8	August	S	895	1264	15640.95
37	8	August	XL	44	44	1122.00
38	8	August	XXL	2	2	71.90
39	9	September	L	1003	1511	29874.10
40	9	September	M	864	1188	18956.75
41	9	September	S	823	1152	14323.35
42	9	September	XL	36	36	918.00
43	9	September	XXL	3	3	107.85
44	10	October	L	978	1485	29338.60
45	10	October	M	878	1253	19992.00
46	10	October	S	830	1101	13554.10
47	10	October	XL	40	42	1071.00
48	10	October	XXL	2	2	71.90
49	11	November	L	1093	1623	32226.20
50	11	November	M	930	1337	21377.75
51	11	November	S	909	1258	15536.05
52	11	November	XL	45	45	1147.50
53	11	November	XXL	3	3	107.85
54	12	December	L	980	1502	29712.20
55	12	December	M	869	1233	19650.50
56	12	December	S	837	1161	14333.50
57	12	December	XL	36	38	969.00
58	12	December	XXL	1	1	35.95

In [58]:
```
import matplotlib.pyplot as plt
import seaborn as sns
```

In [59]:
```
sns.set(style="whitegrid")
plt.rcParams["figure.figsize"] = (10, 6)
```

MONTHLY REVENUE TREND (LINE CHART)

How does revenue change across months?

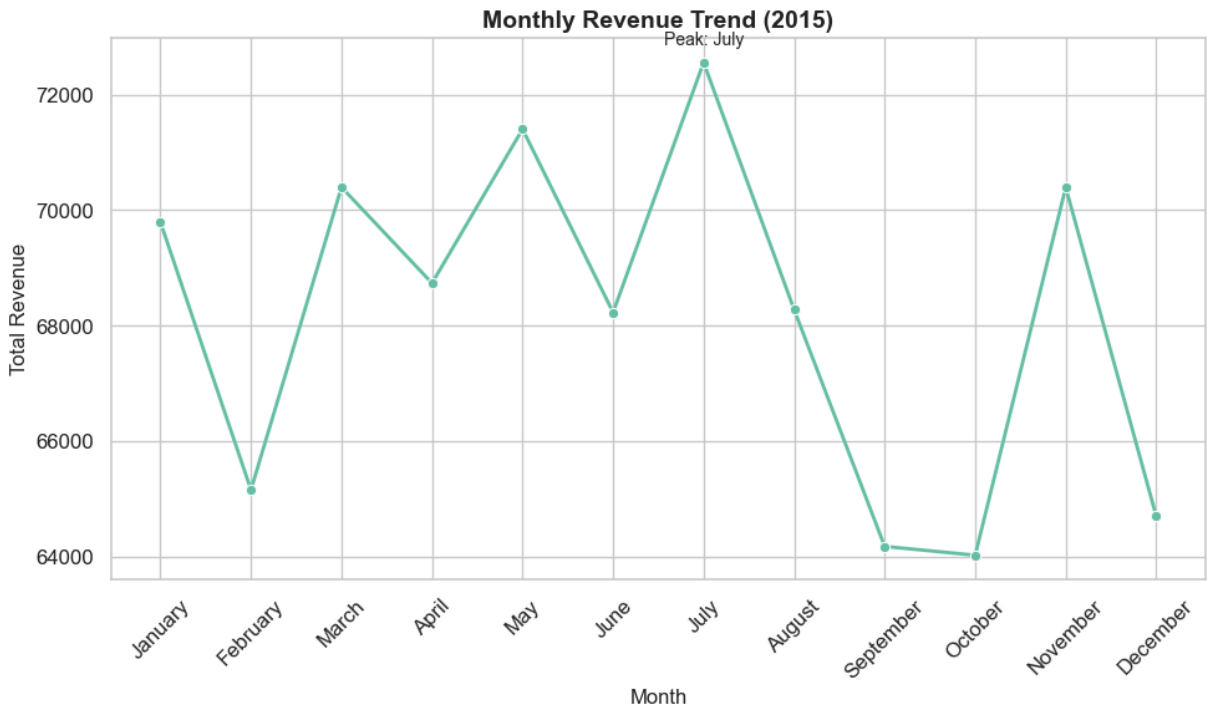
In [60]:
```
monthly_revenue
```

Out[60]:

	month_number	month	total_revenue
0	1	January	69793.30
1	2	February	65159.60
2	3	March	70397.10
3	4	April	68736.80
4	5	May	71402.75
5	6	June	68230.20
6	7	July	72557.90
7	8	August	68278.25
8	9	September	64180.05
9	10	October	64027.60
10	11	November	70395.35
11	12	December	64701.15

In [71]:
```
plt.figure(figsize=(10, 6))

sns.lineplot(
    data=monthly_revenue,
    x="month",
    y="total_revenue",
    marker="o",
    linewidth=2
)

plt.title("Monthly Revenue Trend (2015)")
plt.xlabel("Month")
plt.ylabel("Total Revenue")

plt.xticks(rotation=45)

# Annotate peak month
max_row = monthly_revenue.loc[monthly_revenue["total_revenue"].idxmax()]
```

```
plt.annotate(
    f"Peak: {max_row['month']}",
    xy=(max_row["month"], max_row["total_revenue"]),
    xytext=(0, 10),
    textcoords="offset points",
    ha="center",
    fontsize=10
)

plt.tight_layout()
plt.show()
```

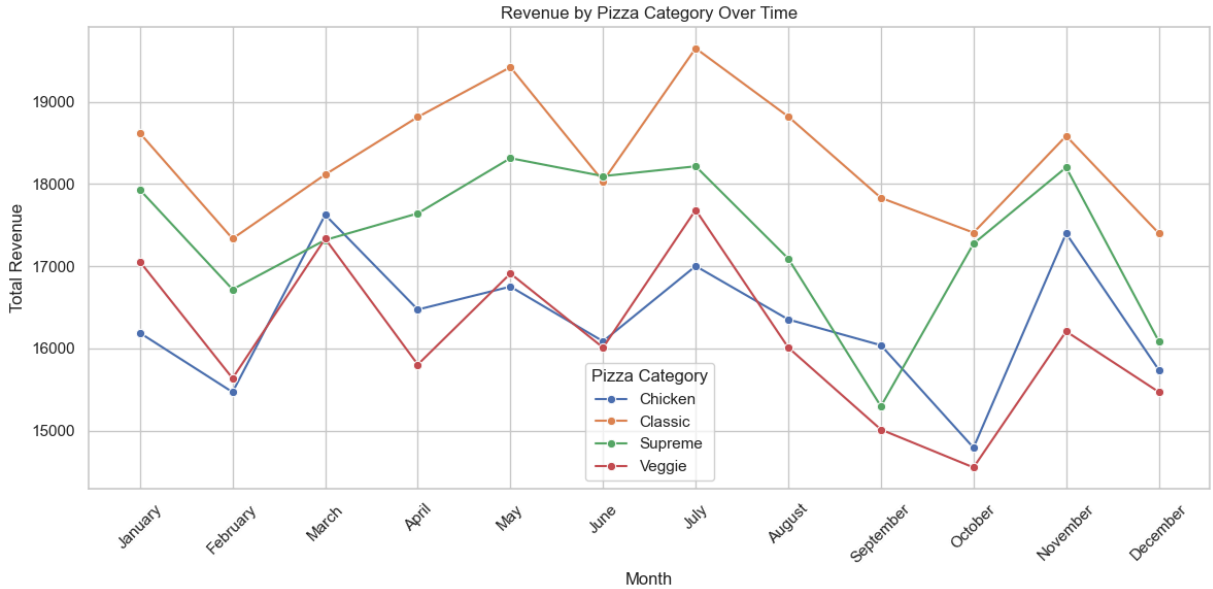


REVENUE BY PIZZA CATEGORY OVER TIME

Plot Category Revenue by Month

In [62]:
```
plt.figure(figsize=(12, 6))

sns.lineplot(
    data=category_monthly,
    x="month",
    y="total_revenue",
    hue="Pizza_category",
    marker="o"
)

plt.title("Revenue by Pizza Category Over Time")
plt.xlabel("Month")
plt.ylabel("Total Revenue")

plt.xticks(rotation=45)
plt.legend(title="Pizza Category")
```

```
plt.tight_layout()
plt.show()
```



TOP 5 PIZZAS PER MONTH (FACET BAR CHART)

USE top_pizzas_by_month

In [63]:
```
top_pizzas_by_month
```

Out[63]:

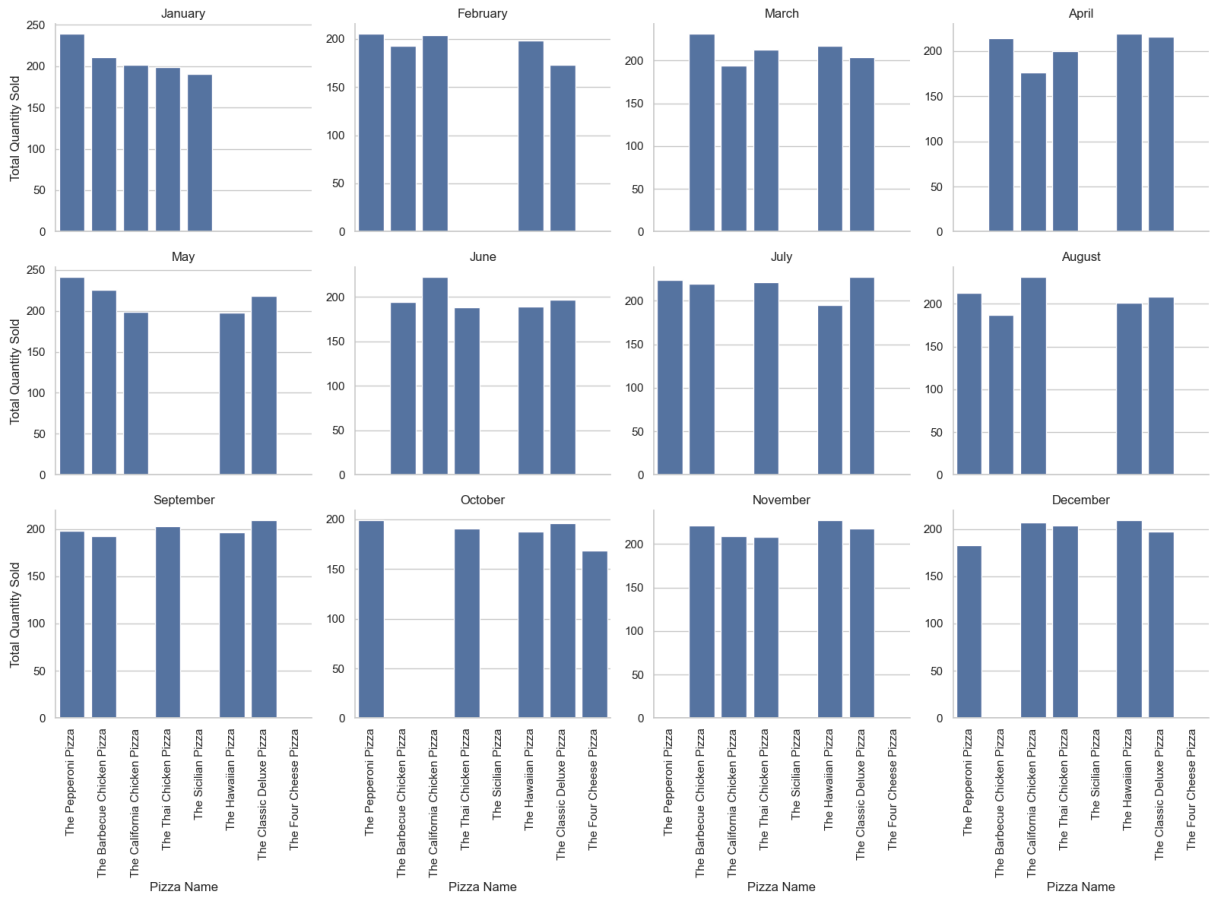
	month_number	month	Pizza_name	total_quantity	total_revenue
20	1	January	The Pepperoni Pizza	239	3001.25
0	1	January	The Barbecue Chicken Pizza	211	3770.25
4	1	January	The California Chicken Pizza	202	3471.50
30	1	January	The Thai Chicken Pizza	199	3657.25
23	1	January	The Sicilian Pizza	191	2987.75
52	2	February	The Pepperoni Pizza	205	2554.25
36	2	February	The California Chicken Pizza	204	3537.00
44	2	February	The Hawaiian Pizza	198	2633.00
32	2	February	The Barbecue Chicken Pizza	193	3416.75
39	2	February	The Classic Deluxe Pizza	173	2705.00
64	3	March	The Barbecue Chicken Pizza	231	4049.25
76	3	March	The Hawaiian Pizza	217	2878.50
94	3	March	The Thai Chicken Pizza	213	3843.75
71	3	March	The Classic Deluxe Pizza	204	3168.00
68	3	March	The California Chicken Pizza	194	3457.50
108	4	April	The Hawaiian Pizza	219	2867.75
103	4	April	The Classic Deluxe Pizza	216	3342.50
96	4	April	The Barbecue Chicken Pizza	214	3788.50
126	4	April	The Thai Chicken Pizza	200	3686.00
100	4	April	The California Chicken Pizza	176	3092.00
148	5	May	The Pepperoni Pizza	241	2998.75
128	5	May	The Barbecue Chicken Pizza	226	3953.50
135	5	May	The Classic Deluxe Pizza	218	3444.00
132	5	May	The California Chicken Pizza	199	3561.25
140	5	May	The Hawaiian Pizza	198	2688.00

	month_number	month	Pizza_name	total_quantity	total_revenue
164	6	June	The California Chicken Pizza	222	3870.50
167	6	June	The Classic Deluxe Pizza	197	3029.00
160	6	June	The Barbecue Chicken Pizza	194	3377.50
172	6	June	The Hawaiian Pizza	189	2563.75
190	6	June	The Thai Chicken Pizza	188	3505.00
199	7	July	The Classic Deluxe Pizza	227	3554.50
212	7	July	The Pepperoni Pizza	224	2819.25
222	7	July	The Thai Chicken Pizza	221	4073.75
192	7	July	The Barbecue Chicken Pizza	219	3784.25
204	7	July	The Hawaiian Pizza	195	2620.25
228	8	August	The California Chicken Pizza	231	3977.25
244	8	August	The Pepperoni Pizza	213	2637.75
231	8	August	The Classic Deluxe Pizza	208	3187.00
236	8	August	The Hawaiian Pizza	201	2678.75
224	8	August	The Barbecue Chicken Pizza	187	3300.25
263	9	September	The Classic Deluxe Pizza	209	3257.50
286	9	September	The Thai Chicken Pizza	203	3736.25
276	9	September	The Pepperoni Pizza	198	2466.75
268	9	September	The Hawaiian Pizza	196	2616.25
256	9	September	The Barbecue Chicken Pizza	192	3380.00
308	10	October	The Pepperoni Pizza	199	2471.00
295	10	October	The Classic Deluxe Pizza	196	3075.50
318	10	October	The Thai Chicken Pizza	191	3503.25
300	10	October	The Hawaiian Pizza	188	2514.75
297	10	October	The Four Cheese Pizza	169	2870.35
332	11	November	The Hawaiian Pizza	227	2952.75
320	11	November	The Barbecue Chicken Pizza	221	3881.75

	month_number	month	Pizza_name	total_quantity	total_revenue
327	11	November	The Classic Deluxe Pizza	218	3379.50
324	11	November	The California Chicken Pizza	209	3692.75
350	11	November	The Thai Chicken Pizza	208	3776.00
364	12	December	The Hawaiian Pizza	209	2816.75
356	12	December	The California Chicken Pizza	207	3575.25
382	12	December	The Thai Chicken Pizza	204	3813.00
359	12	December	The Classic Deluxe Pizza	197	3096.50
372	12	December	The Pepperoni Pizza	183	2293.00

In [66]:
```
top_pizzas_by_month = (
    top_pizzas_by_month
    .sort_values(["month_number", "total_quantity"], ascending=[True, False])
)
```

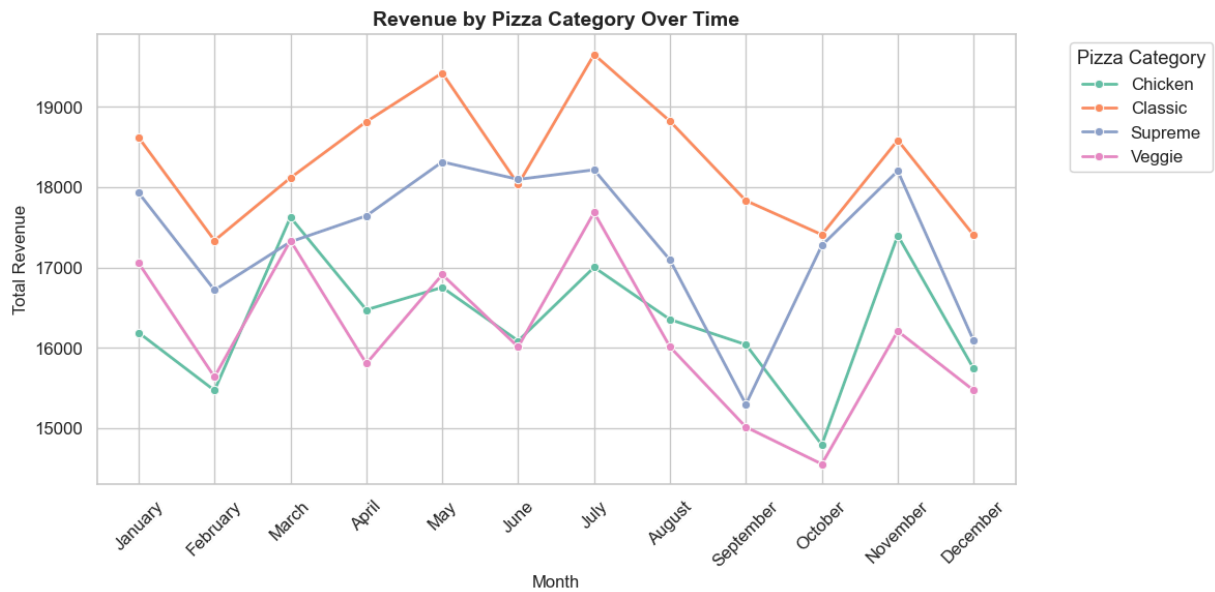
In [67]:
```
g = sns.catplot(
    data=top_pizzas_by_month,
    x="Pizza_name",
    y="total_quantity",
    col="month",
    col_wrap=4,
    kind="bar",
    height=4,
    sharey=False
)

g.set_titles("{col_name}")
g.set_axis_labels("Pizza Name", "Total Quantity Sold")

for ax in g.axes.flat:
    ax.tick_params(axis="x", rotation=90)

plt.tight_layout()
plt.show()
```



PIZZA SIZE TRENDS OVER TIME (ORDERS)

Plot Orders by Pizza Size Over Time

In [73]:
```
plt.figure(figsize=(12, 6))

sns.lineplot(
    data=category_monthly,
    x="month",
    y="total_revenue",
    hue="Pizza_category",
    marker="o",
    linewidth=2
)

plt.title("Revenue by Pizza Category Over Time")
plt.xlabel("Month")
plt.ylabel("Total Revenue")

plt.xticks(rotation=45)
plt.legend(title="Pizza Category", bbox_to_anchor=(1.05, 1), loc="upper left")

plt.tight_layout()
plt.show()
```



TOP 10 PIZZAS (OVERALL) – HORIZONTAL BAR

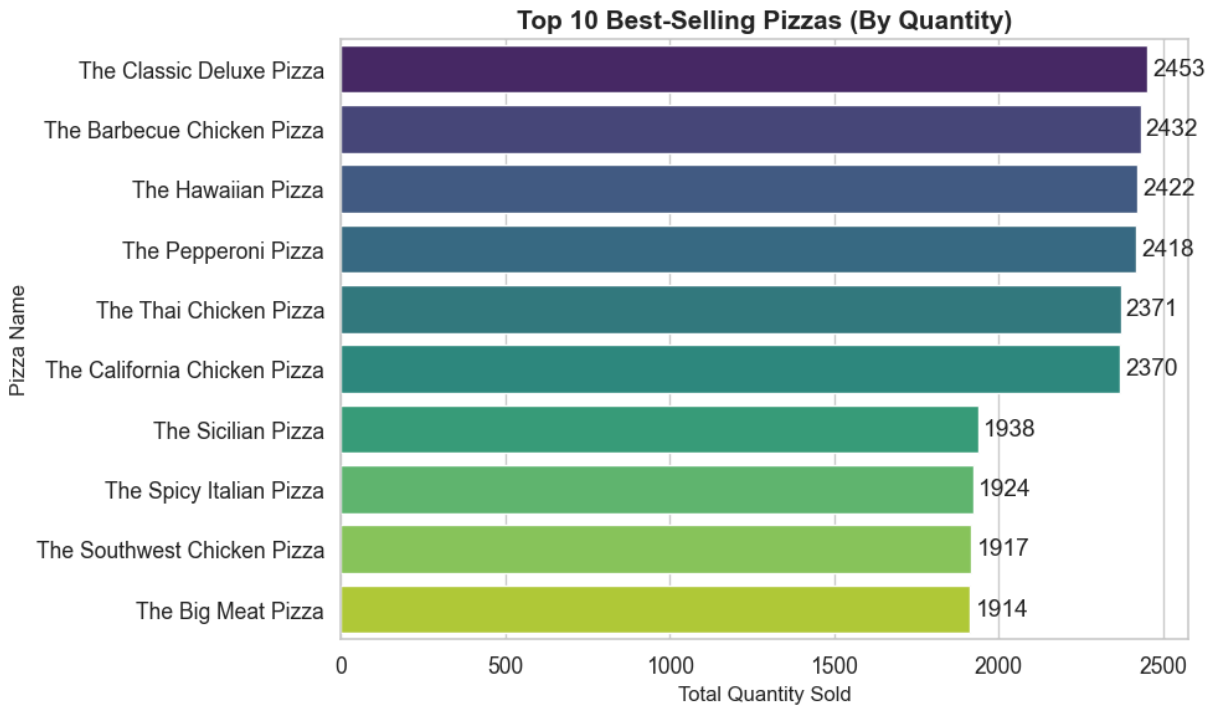
In [12]:
```
top_10_pizzas = (
    df.groupby("Pizza_name", as_index=False)
      .agg(
          total_quantity=("Quantity", "sum"),
          total_revenue=("Total_price", "sum")
      )
      .sort_values("total_quantity", ascending=False)
      .head(10)
)

top_10_pizzas
```

Out[12]:

	Pizza_name	total_quantity	total_revenue
7	The Classic Deluxe Pizza	2453	38180.50
0	The Barbecue Chicken Pizza	2432	42768.00
12	The Hawaiian Pizza	2422	32273.25
20	The Pepperoni Pizza	2418	30161.75
30	The Thai Chicken Pizza	2371	43434.25
4	The California Chicken Pizza	2370	41409.50
23	The Sicilian Pizza	1938	30940.50
26	The Spicy Italian Pizza	1924	34831.25
25	The Southwest Chicken Pizza	1917	34705.75
1	The Big Meat Pizza	1914	22968.00

In [13]:
```
plt.figure(figsize=(10, 6))
```

```
ax = sns.barplot(
    data=top_10_pizzas,
    y="Pizza_name",
    x="total_quantity",
    hue="Pizza_name",
    palette="viridis",
    legend=False
)

# ✅ ADD DATA LABELS
for container in ax.containers:
    ax.bar_label(container, padding=3)

plt.title("Top 10 Best-Selling Pizzas (By Quantity)")
plt.xlabel("Total Quantity Sold")
plt.ylabel("Pizza Name")

plt.tight_layout()
plt.show()
```



Key Insights

- Customers purchase an average of ~2.3 pizzas per order
- Certain pizza categories exhibit strong seasonal behavior
- Medium and Large sizes dominate order volume
- Revenue concentration suggests opportunities for targeted promotions ``